



Discord OAuth 로그인 완전 정복 가이드

SteamIron 백엔드 — feat/discord-login 브랜치에서 구현한 내용의 학습 문서

2026-07-15 작성 · Spring Boot 4.1 · Java 21 · 사전지식이 거의 없는 상태에서 읽는 것을 기준으로 작성됨

목차

1. 이 문서를 읽는 법
2. 기초 다지기 — 클라이언트, 서버, HTTP
3. 로그인 이론 — OAuth2는 어떤 문제를 푸는가
4. JWT와 Refresh Token — 로그인 "상태"를 어떻게 기억하나
5. 스프링(Spring) 기초 — 코드를 읽기 위한 최소한의 지식
6. 이 프로젝트의 구조
7. 작성한 코드 전체 해설 (파일별, 한 줄씩)
8. 설정 파일(application.yaml) 해설
9. 구현하면서 실제로 겪은 문제와 해결 과정
10. 어떻게 검증했나
11. 남은 할 일 — 직접 해야 하는 것들
12. 용어 사전 (한 페이지 요약)

1. 이 문서를 읽는 법

이 문서는 "Discord 계정으로 우리 서비스에 로그인하는 기능"을 만들면서 작성된 학습 자료입니다. 크게 **이론(2~5장)** → **실제 코드(6~8장)** → **경험담(9~10장)** 순서로 되어 있습니다.

코드가 어렵게 느껴지면 5장(스프링 기초)까지만 먼저 정독하고, 6장부터는 실제 파일을 IDE로 열어놓고 대조하면서 읽는 것을 추천합니다. 초록색 상자(**용어**)는 처음 나오는 용어의 정의, 파란 상자는 핵심 개념, 주황 상자는 실수하기 쉬운 함정입니다.

2. 기초 다지기 — 클라이언트, 서버, HTTP

2-1. 클라이언트와 서버

웹 서비스는 크게 두 부분으로 나뉩니다.

클라이언트 (프론트엔드)		서버 (백엔드)
무엇	유저의 브라우저에서 실행되는 화면 (React 등)	회사 컴퓨터에서 24시간 돌아가는 프로그램 (우리의 Spring 서버)
역할	화면을 그리고, 유저 입력을 받아 서버에 요청	요청을 받아 DB를 읽고 쓰고, 결과를 응답
신뢰도	유저 손에 있으므로 믿으면 안 됨 (코드가 다 보이고 조작 가능)	우리가 통제하므로 비밀값(secret)을 보관 가능

이 구분이 중요한 이유: 뒤에서 나올 `client_secret` 같은 비밀값을 왜 반드시 백엔드에만 뒀야 하는지가 여기서 결정됩니다. 브라우저에서 실행되는 코드는 F12(개발자 도구)만 눌러도 전부 들여다볼 수 있기 때문입니다.

2-2. HTTP — 클라이언트와 서버가 대화하는 규칙

HTTP — 클라이언트와 서버가 주고받는 메시지의 형식을 정한 규칙(프로토콜). 브라우저 주소창에 치는 모든 것, 앱이 서버와 통신하는 것 대부분이 HTTP입니다.

HTTP **요청(request)** 은 실제로는 아래처럼 생긴 텍스트입니다. 딱 3부분입니다.

```

POST /api/auth/login/discord HTTP/1.1
Host: api.steamiron.com
Content-Type: application/json

{ "code": "Nhhv..." }

```

← ① 요청 라인: 메서드 + 경로
 ← ② 헤더: 부가 정보 (여러 줄 가능)
 ← ② 헤더
 ← (빈 줄이 헤더와 본문의 경계)
 ← ③ 본문(body): 실제 데이터

① 메서드 — 이 요청이 어떤 종류의 행동인지

메서드	의미	데이터를 실는 곳	이번 구현에서의 예
GET	조회 (읽기)	URL 뒤 <code>?a=1&b=2</code>	Discord 유저 정보 조회
POST	제출/생성 (처리해달라)	본문(body)	로그인, 토큰 재발급, 로그아웃

POST를 쓰는 이유 중 하나는 **보안**입니다. URL은 서버 로그·브라우저 방문기록에 그대로 남지만 본문은 남지 않습니다. 비밀값은 본문에 실어야 합니다.

② 헤더 — 요청/응답에 붙는 부가 정보

헤더는 `이름: 값` 형태의 부가 정보입니다. 이번 구현에서 실제로 쓴 헤더는 두 개입니다.

Content-Type 헤더 — "내 본문(body)이 어떤 표기법으로 적혀 있는지" 알려주는 라벨. 같은 데이터도 여러 형식으로 적을 수 있어서, 받는 쪽이 해석하려면 이 라벨이 필요합니다. 틀리면 서버가 본문을 못 읽어 400 에러가 납니다.

형식 1) application/json - 요즘 API의 표준. 중괄호와 따옴표.

```
{ "grant_type": "authorization_code", "code": "Nh hv" }
```

형식 2) application/x-www-form-urlencoded - 옛날 HTML <form> 방식. key=value를 &로 연결.

```
grant_type=authorization_code&code=Nh hv
```

내용은 같고 표기법만 다릅니다. **OAuth2 표준은 토큰 요청을 반드시 형식 2로 보내라고 정해놨습니다.** 그래서 Discord에 보내는 요청만 form 형식이고, 우리 프론트↔백엔드 사이는 JSON을 씁니다.

Authorization 헤더 — "나 인증된 사용자야"를 증명하는 토큰을 싣는 헤더. `Authorization: Bearer` 토큰 문자열 형태로 쓰며, `Bearer` (소지자)는 "이 토큰을 가진 자를 인증하라"는 뜻의 관용 표기입니다.

③ 상태 코드 — 응답의 결과 요약

서버의 **응답(response)** 에는 항상 3자리 숫자 상태 코드가 붙습니다. 이번 구현에서 등장하는 것들:

코드	이름	의미	이번 구현에서 언제
200	OK	성공	로그인 성공, 재발급 성공
400	Bad Request	요청 형식이 잘못됨	본문 JSON이 깨졌을 때 등
401	Unauthorized	인증 실패 (누군지 확인 안 됨)	토큰 없음/만료/위조, code 무효
403	Forbidden	인증은 됐지만 권한 없음	(일반 유저가 관리자 API 접근 등)
500	Internal Server Error	서버 코드에서 예외 발생	버그

2-3. API란

API — 서버가 외부에 공개한 "요청을 받는 창구"의 목록. " `POST /api/auth/login/discord` 로 `{code}` 를 보내면 토큰을 응답한다"처럼 **주소 + 메서드 + 요청형식 + 응답형식**의 약속 하나하나를 API 또는 엔드포인트(endpoint)라고 부릅니다.

이번에 우리가 만든 API는 3개입니다. 이 표가 이 문서 전체의 요약이기도 합니다.

API	받는 것	돌려주는 것	하는 일
<code>POST /api/auth/login/discord</code>	<code>{ code }</code>	accessToken, refreshToken, 유저정보	Discord 로그인 (없으면 자동 회원가입)
<code>POST /api/auth/reissue</code>	<code>{ refreshToken }</code>	새 accessToken + 새 refreshToken	만료된 accessToken 재발급
<code>POST /api/auth/logout</code>	<code>{ refreshToken }</code>	(없음)	DB의 refreshToken 삭제

3. 로그인 이론 — OAuth2는 어떤 문제를 푸는가

3-1. 왜 OAuth인가

로그인을 직접 만들면 우리가 유저의 비밀번호를 받아서 안전하게 저장하고(암호화), 유출도 막아야 합니다. 부담이 큼니다. 그래서 "이 사람이 누구인지 확인하는 일"을 **이미 잘하는 곳(Discord)에 맡기는 것**이 소셜 로그인이고, 그때 쓰는 표준 규격이 OAuth2입니다.

OAuth2의 핵심 아이디어 — 유저의 Discord 비밀번호는 오직 Discord만 받는다. 우리 서비스는 비밀번호 대신 "**Discord가 인증을 끝냈다는 증표(code)**"를 받고, 그 증표를 Discord에 제출해서 "이 사람 누구예요?"라고 물어 본다.

3-2. 전체 흐름 — 요청 6단계

로그인 버튼을 누른 순간부터 로그인 완료까지, 등장인물 3명(브라우저, 우리 백엔드, Discord) 사이에 일어나는 일 전부입니다.

[①] 브라우저 —————▶ Discord

"로그인하고 싶어" (인증 페이지로 이동)

`https://discord.com/oauth2/authorize?client_id=..&redirect_uri=..&response_type=code&scope=identify email`

유저가 Discord 화면에서 로그인하고 [승인] 클릭

(비밀번호는 Discord 안에서만 오감 - 우리는 절대 못 봄)

[②] Discord —————▶ 브라우저

`redirect_uri`로 돌려보내면서 `code`를 붙여줌

`http://localhost:5173/auth/callback?code=Nhhv...`

[③] 브라우저 —————▶ 우리 백엔드

`POST /api/auth/login/discord { "code": "Nhhv..." }`

※ 프론트가 하는 일은 여기까지!

[④] 우리 백엔드 —————▶ Discord

`POST https://discord.com/api/oauth2/token (form 형식)`

`grant_type=authorization_code&code=..&redirect_uri=..`

`&client_id=..&client_secret=..` ← 비밀값은 여기서 처음 등장

◀— 응답: `{ "access_token": "6qrZ...", ... }`

[⑤] 우리 백엔드 —————▶ Discord

`GET https://discord.com/api/users/@me`

`Authorization: Bearer 6qrZ...`

◀— 응답: `{ "id": "8035111...", "username": "slug", "avatar": "...", "email": ".." }`

[⑥] 우리 백엔드 내부 처리 후 —————▶ 브라우저

DB에서 `discordId`로 유저 찾기 (없으면 자동 회원가입)

우리 서비스 전용 JWT 발급

◀— 응답: `{ "accessToken": "eyJhb...", "refreshToken": "...", "username": "slug" }`

3-3. 각 단계에서 오가는 값들의 정체

client_id / client_secret — Discord Developer Portal에서 우리 앱을 등록하면 발급되는 한 쌍. `client_id`는 우리 앱의 이름표(공개되도 됨), `client_secret`은 우리 앱의 비밀번호(**백엔드만 알아야 함**). ④에서 시크릿을 함께 제출해야 Discord가 "code를 제출한 게 진짜 그 서비스"임을 믿습니다.

redirect_uri — 인증이 끝난 유저를 돌려보낼 주소. Developer Portal에 미리 등록해야 하며, ①에서 보낸 값·④에서 보낸 값·포털에 등록한 값 세 곳이 글자 하나까지 일치해야 합니다. 공격자가 code를 자기 사이트로 빼돌리는 것을 막는 장치입니다.

code (인증 코드) — "이 유저가 방금 Discord 인증에 성공했다"는 **1회용 증표**. 수명 몇 분. 한 번 사용하면 즉시 무효. 이것만으로는 아무것도 못 하고, ④에서 `client_secret`과 함께 제출해야 의미가 생깁니다.

scope — 우리가 유저에 대해 요청하는 정보의 범위. `identify` (id·닉네임·아바타), `email` (이메일). 유저가 승인 화면에서 "이 서비스가 이런 정보를 요구합니다"로 보게 되는 부분입니다.

Discord access_token — Discord API를 호출할 수 있는 열쇠. ⑤에서 유저 정보를 조회할 때 딱 한 번 쓰고 버립니다. **우리 서비스의 로그인 토큰이 아닙니다.** 로그인 완료 후에는 Discord가 전혀 관여하지 않습니다.

grant_type — OAuth2에서 토큰을 받는 방법이 여러 가지(`authorization_code` , `refresh_token` , `client_credentials`)인데, 토큰 발급 API 주소는 하나뿐이라 "어떤 방법으로 받으려는지"를 명시하는 필드. 우리는 `code`를 제출하는 방식이므로 항상 `authorization_code` 고정.

왜 ④를 브라우저가 직접 안 하고 백엔드가 하나? — ④에는 `client_secret`이 필요한데, 브라우저(프론트 코드)에 시크릿을 넣으면 F12로 누구나 볼 수 있습니다. 시크릿이 유출되면 공격자가 우리 서비스 행세를 할 수 있습니다. 그래서 **code→토큰 교환은 반드시 백엔드에서 합니다.**

4. JWT와 Refresh Token — 로그인 "상태"를 어떻게 기억하나

4-1. HTTP는 기억력이 없다

HTTP 요청은 서로 완전히 독립적입니다. 1분 전에 로그인한 유저가 다시 요청을 보내도, 서버 입장에서는 처음 보는 요청입니다. 그래서 "나 아까 로그인한 그 사람이야"를 매 요청마다 증명할 방법이 필요합니다. 두 가지 방식이 있습니다.

세션 방식		토큰(JWT) 방식 ← 우리가 쓰는 것
원리	서버가 "로그인한 사람 명단"을 메모리/DB에 보관, 유저에게 킴 명단 번호(세션 ID)만 줌	서버는 아무것도 보관 안 함. 유저에게 위조 불가능한 증명서(JWT) 를 통째로 줌
검증	매 요청마다 명단 조회	매 요청마다 증명서의 서명만 검사 (DB 조회 불필요)
장점	서버가 강제 로그아웃 가능	서버 확장에 유리, 별도 저장소 불필요
단점	서버가 여러 대면 명단 공유 문제	발급 후엔 만료 전까지 무효화 불가

4-2. JWT의 생김새

JWT (JSON Web Token) — 점(.)으로 구분된 3부분짜리 문자열. **헤더.내용물.서명**

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiIzIiwiaXNjaXhwIjoxNzUyNTE2MDAwfQ . 4Xk9fJq...
```

[헤더] 어떤 서명 알고리즘을 썼나 → {"alg":"HS256"}
[내용물] 토큰에 담긴 정보 (claims라고 부름) → {"sub":"3","exp":1752516000}
sub(subject) = 이 토큰의 주인 = 우리 DB의 userId
exp(expiration) = 만료 시각
[서명] 헤더+내용물을 서버만 아는 비밀키로 암호 계산한 값

서명이 핵심입니다. 내용물은 사실 누구나 열어볼 수 있습니다(암호화가 아니라 인코딩). 하지만 내용물을 한 글자라도 바꾸면 서명이 안 맞게 되고, 서명을 새로 만들려면 서버의 비밀키가 필요합니다. 즉 **"위조는 불가능하지만 내용은 보이는 증명서"**입니다. 그래서 JWT 안에 비밀정보를 넣으면 안 되고, userId 정도만 넣습니다.

4-3. Refresh Token — 유효기간 딜레마의 해법

JWT는 발급하면 무효화가 불가능하므로 유효기간을 정할 때 딜레마가 생깁니다.

길게(2주) → 편하지만, 도난당하면 도둑이 2주 동안 유저 행세 (막을 방법 없음)
짧게(30분) → 안전하지만, 유저가 30분마다 로그인 풀림

해법: 토큰을 2개로 나눕니다.

	accessToken	refreshToken
수명	짧게 — 30분	길게 — 14일
용도	매 API 요청의 인증	accessToken 재발급 한 가지뿐
형태	JWT (서버 저장 안 함)	랜덤 문자열(UUID), DB에 저장
서버가 무효화	불가능 (그래서 짧게)	가능 — DB에서 지우면 끝

평소: 프론트 —(accessToken)—▶ 백엔드 ✓ 통과
30분 후: 프론트 —(만료된 accessToken)—▶ 백엔드 ✗ 401
자동으로: 프론트 —POST /reissue {refreshToken}—▶ 백엔드
 백엔드: DB에서 refreshToken 찾기 → 만료 검사 → 통과!
 ◀— 새 accessToken + 새 refreshToken
 프론트가 원래 요청 재시도 → 유저는 아무것도 못 느낌

결과적으로 유저는 14일 동안 로그인이 유지되는데, 실제로 돌아다니는 accessToken은 30분짜리입니다.

토큰 로테이션(rotation) — 재발급할 때 refreshToken도 새 값으로 갈아끼우는 전략. 한 번 쓴 refreshToken을 계속 재사용하게 두는 것보다 탈취 피해가 줄어듭니다. 우리 구현이 이 방식입니다.

refreshToken을 DB에 저장하는 이유 — 서버가 강제로 끊을 수단을 갖기 위해서. 로그아웃 = DB에서 삭제. 탈취 의심 = DB에서 삭제. 재발급 요청이 왔는데 DB에 없다 = 거부하고 재로그인 요구.

정책 결정: 유저당 refreshToken 1개 — 우리는 한 유저가 로그인하면 기존 refreshToken을 새 값으로 교체합니다. 장점: 테이블에 쓰레기가 안 쌓임, 관리 단순. 트레이드오프: PC에서 로그인하면 폰의 로그인이 풀림. 여러 기기 동시 로그인을 허용하려면 유저당 여러 행을 허용하고, 만료된 행을 배치(batch)로 주기 삭제해야 합니다 (이 프로젝트에 app-batch-server가 있는 이유 중 하나가 될 수 있음).

헛갈림 주의 — ④에서 Discord가 준 응답에도 refresh_token 이 있습니다. 그건 **Discord API용**이고, 4장의 refreshToken은 **우리 서비스용**입니다. 이름만 같은 완전 별개입니다. 우리는 Discord 토큰을 한 번 쓰고 버리므로 Discord의 refresh_token은 쓸 일이 없습니다.

5. 스프링(Spring) 기초 — 코드를 읽기 위한 최소한의 지식

5-1. 스프링과 스프링부트

Spring Framework — 자바로 서버를 만들 때 반복되는 뼈대 작업(요청 받기, DB 연결, 객체 관리...)을 대신 해주는 거대한 라이브러리 모음. **Spring Boot**는 스프링의 복잡한 설정을 자동화해서 "실행하면 바로 서버가 뜨는" 형태로 포장한 것. 이 프로젝트는 Spring Boot 4.1을 씁니다.

5-2. 어노테이션(@)이란

어노테이션(annotation) — 클래스·메서드·필드에 붙이는 `@이름` 형태의 표식. 코드 자체의 동작을 바꾸는 게 아니라, 스프링(또는 다른 도구)이 읽고 "아 이걸 이렇게 취급하라는 거구나"라고 알아듣는 라벨입니다. 예: `@RestController` 가 붙은 클래스 = "HTTP 요청을 받는 창구로 등록해줘".

5-3. 빈(Been)과 의존성 주입(DI) — 스프링의 심장

스프링 없이 자바를 쓰면 객체를 이렇게 직접 만들어 연결해야 합니다.

```
// 스프링이 없다면... 누군가 이 조립을 전부 손으로 해야 함
JwtProvider jwt = new JwtProvider("비밀키", 1800000);
DiscordOAuthClient discord = new DiscordOAuthClient("id", "secret", "uri");
AuthService service = new AuthService(discord, userRepo, tokenRepo, jwt);
AuthController controller = new AuthController(service);
```

빈(Been) — 스프링이 대신 만들어서 보관·관리해주는 객체. `@Component`, `@Service`, `@RestController`, `@Configuration` + `@Bean` 등이 붙은 클래스는 서버 시작 시 스프링이 **알아서 한 개씩 만들어서 컨테이너(보관함)에 등록**합니다.

의존성 주입(DI, Dependency Injection) — 어떤 빈이 다른 빈을 필요로 하면(=의존하면), 스프링이 보관함에서 꺼내 **생성자의 파라미터로 자동으로 넣어주는 것**. 위의 손조립 코드를 스프링이 전부 대신 해주는 셈입니다. 우리 코드에서 생성자에 `final` 필드가 있고 `@RequiredArgsConstructor` (롬복)가 붙어 있으면 "이것들을 주입받겠다"는 뜻입니다.

이번 구현의 조립도 — 스프링이 서버 시작 시 자동으로 이렇게 연결합니다.

```
AuthController ← AuthService ← (DiscordOAuthClient, UserRepository, RefreshTokenRepository, JwtProvider)
```

5-4. 계층 구조 — Controller / Service / Repository

스프링 백엔드는 관례적으로 역할을 3계층으로 나눕니다. 요청 하나가 위에서 아래로 흘렀다가 되돌아옵니다.

계층	역할	대표 어노테이션	우리 파일
Controller	HTTP를 받는 창구. 요청을 자바 객체로 변환하고 응답을 돌려줌. 로직은 안 씀	@RestController	AuthController
Service	비즈니스 로직(실제 일 처리). "로그인이란 무엇을 하는 것인가"가 여기 적힘	@Service	AuthService
Repository	DB 읽기/쓰기 전담	JpaRepository 상속	UserRepository 등

이렇게 나누는 이유: 각 층을 따로 수정·테스트할 수 있고, 코드를 찾기 쉬워집니다. (Discord와 통신하는 `DiscordOAuthClient` 는 외부 API 호출 전담이라 별도로 client로 분류했습니다.)

5-5. JPA — 자바 객체와 DB 테이블을 연결

JPA / Hibernate — "자바 클래스 ↔ DB 테이블"을 자동으로 오가게 해주는 기술. SQL을 직접 안 써도 `userRepository.save(user)` 라고 쓰면 INSERT문이 실행됩니다. JPA는 표준 규격이고 Hibernate는 그 구현체(실제 엔진)입니다.

엔티티(Entity) — DB 테이블과 1:1로 대응되는 자바 클래스. `@Entity` 를 붙입니다. 클래스 = 테이블, 필드 = 컬럼, 인스턴스 하나 = 행(row) 하나.

ddl-auto: update — 서버가 시작될 때 엔티티 클래스를 보고 DB에 없는 테이블/컬럼을 **자동으로 만들어주는** 설정. 개발 단계에서 편리합니다. (운영에서는 위험해서 보통 끕니다 — 실수로 스키마가 바뀔 수 있어서.)

쿼리 메서드 — Repository 인터페이스에 `findByDiscordId(String discordId)` 라고 **메서드 이름만 규칙에 맞게 선언하면**, 스프링이 이름을 해석해서 `SELECT * FROM users WHERE discord_id = ?` 쿼리를 자동 생성해주는 기능. 구현 코드를 한 줄도 안 씁니다.

5-6. 이 문서에 나오는 롬복(Lombok) 어노테이션

롬복(Lombok) — 자바의 반복 코드(getter, 생성자 등)를 컴파일 시점에 자동 생성해주는 라이브러리.

어노테이션	자동 생성해주는 것
@Getter	모든 필드의 <code>getXxx()</code> 메서드
@NoArgsConstructor	파라미터 없는 기본 생성자
@RequiredArgsConstructor	<code>final</code> 필드들을 파라미터로 받는 생성자 (→ DI에 사용됨)
@Builder	<code>User.builder().username("slug").build()</code> 식으로 읽기 좋게 객체를 만드는 코드

6. 이 프로젝트의 구조

6-1. 멀티모듈

이 레포는 하나의 프로젝트를 3개의 모듈로 나눈 **멀티모듈** 구조입니다.

```
SteamIron-BE/
├─ module-domain-rdb/    # 엔티티 + Repository만 모아둔 공용 라이브러리 모듈
│                         # (실행되는 서버가 아님. 다른 모듈이 가져다 씀)
├─ app-api-server/       # 실제 API 서버 (우리가 이번에 작업한 곳)
└─ app-batch-server/     # 주기 작업(배치) 서버 (크롤링 등)
```

왜 엔티티를 별도 모듈에? — User 테이블은 API 서버도 쓰고 배치 서버도 씁니다. 각자 정의하면 두 정의가 어긋나는 사고가 나므로, 한 곳(module-domain-rdb)에 두고 양쪽이 공유합니다. `app-api-server/build.gradle` 의 `implementation project(':module-domain-rdb')` 가 "그 모듈을 가져다 쓴다"는 선언입니다.

6-2. 이번에 만든 파일 전체 지도

```
module-domain-rdb/src/main/java/norimaets/moduledomainrdb/
├─ user/
│   ├── entity/User.java           # 유저 테이블
│   └─ repository/UserRepository.java # 유저 DB 조회/저장
└─ auth/
    ├── entity/RefreshToken.java    # refresh token 테이블
    └─ repository/RefreshTokenRepository.java

app-api-server/src/main/java/norimaets/appapiserver/
├─ domain/auth/
│   ├── controller/AuthController.java # API 창구 3개
│   ├── service/AuthService.java       # 로그인/재발급/로그아웃 로직
│   ├── client/DiscordOAuthClient.java # Discord와 통신 (OAuth)
│   └─ dto/                             # 요청/응답 데이터 상자 6개
│       ├── DiscordLoginRequest.java    #   프론트 → 우리: { code }
│       ├── RefreshTokenRequest.java    #   프론트 → 우리: { refreshToken }
│       ├── LoginResponse.java          #   우리 → 프론트: 토큰+유저정보
│       ├── ReissueResponse.java        #   우리 → 프론트: 새 토큰들
│       ├── DiscordTokenResponse.java   #   Discord → 우리: access_token 등
│       └─ DiscordUserResponse.java     #   Discord → 우리: id, username 등
└─ global/security/
    ├── JwtProvider.java               # JWT 만들기/검증하기
    ├── JwtAuthenticationFilter.java   # 매 요청 JWT 검사하는 문지기
    └─ SecurityConfig.java             # 보안 규칙 설정
```

6-3. 로그인 요청 하나가 코드를 통과하는 경로

POST /api/auth/login/discord { "code": "..."} }

|

▼ (스프링 시큐리티 필터들 → JwtAuthenticationFilter는 토큰 없으니 그냥 통과

| → SecurityConfig의 규칙상 /api/auth/** 는 permitAll이라 인증 없이 진입 허용)

▼

AuthController.loginWithDiscord(request) # JSON → DiscordLoginRequest 자동 변환

▼

AuthService.loginWithDiscord(code)

└─▶ DiscordOAuthClient.exchangeCode(code) # ㉔ code → Discord access_token

└─▶ DiscordOAuthClient.fetchUser(accessToken) # ㉕ 유저 정보 조회

└─▶ UserRepository.findByDiscordId(...) # ㉖ 있으면 갱신, 없으면 save() = 자동 회원가입

└─▶ JwtProvider.createAccessToken(userId) # accessToken(JWT) 발급

└─▶ RefreshTokenRepository (save 또는 rotate) # refreshToken 발급+DB 저장

▼

LoginResponse 반환 → 스프링이 JSON으로 변환 → 프론트에 응답

7. 작성한 코드 전체 해설

7-1. User.java — 유저 테이블

```
@Entity
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@Table(name = "users") // user는 PostgreSQL 예약어라서 users로 지정
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String discordId;

    @Column(nullable = false)
    private String username;

    private String avatarUrl;
    private String email;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;
    ...
}
```

코드	의미와 이유
@Entity	"이 클래스는 DB 테이블이다"라고 JPA에 알림. 서버 시작 시 이걸 보고 테이블을 만듦(ddl-auto)
@Table(name="users")	테이블 이름 지정. user 는 PostgreSQL의 예약어(DB가 자체적으로 쓰는 단어)라 그대로 쓰면 SQL 에러가 나서 users 로 지정
@Id + @GeneratedValue(IDENTITY)	기본키(행마다 고유 번호). IDENTITY = DB가 1, 2, 3... 자동 증가로 채번
discordId 에 unique=true	Discord가 부여한 절대 안 바뀌는 고유번호. 유저 식별은 항상 이걸로 함 . username은 유저가 바꿀 수 있어서 식별자로 쓰면 안 됨. unique 제약으로 같은 Discord 계정의 중복 가입을 DB 차원에서 차단
nullable = false	"이 컬럼은 비어 있으면 안 된다"는 DB 제약(NOT NULL)
updatable = false	가입 시각(createdAt)은 한 번 저장되면 수정 불가
@NoArgsConstructor(PROTECTED)	JPA는 기술적인 이유로 기본 생성자를 요구함. 하지만 아무나 new User() 로 빈 객체를 만들면 위험하니 protected로 잠가서 JPA만 쓰게 함
updateProfile(...) 메서드	setter를 다 열어두는 대신, "프로필 갱신"이라는 의도가 드러나는 메서드만 공개. Discord에서 닉네임/아바타를 바꿨을 수 있어서 로그인할 때마다 이걸로 갱신함

7-2. RefreshToken.java — refresh token 테이블

```

@Entity
@Table(name = "refresh_tokens")
public class RefreshToken {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String token;          // 토큰 값 자체 (UUID 랜덤 문자열)

    @Column(nullable = false, unique = true)
    private Long userId;          // 누구의 토큰인지

    @Column(nullable = false)
    private LocalDateTime expiryDate; // 만료 시각

    public void rotate(String newToken, LocalDateTime newExpiryDate) { ... }
    public boolean isExpired() { return expiryDate.isBefore(LocalDateTime.now()); }
}

```

- `userId` 가 `unique` = "유저당 refreshToken 1개" 정책을 DB 제약으로 강제한 것 (4-3의 정책 결정 참고)
- `rotate()` = 재발급 때 토큰 값과 만료시각을 새것으로 교체하는 메서드 (토큰 로테이션)
- `isExpired()` = 만료 여부 판단을 엔티티 스스로 하게 해서, 서비스 코드가 깔끔해짐

7-3. Repository 2개 — DB 접근

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByDiscordId(String discordId);
}

public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {
    Optional<RefreshToken> findByToken(String token);
    Optional<RefreshToken> findByUserId(Long userId);
    void deleteByToken(String token);
}
```

- `JpaRepository<User, Long>` 을 상속(extends)하면 `save()`, `findById()`, `delete()` 등 기본 DB 작업이 **전부 공짜**로 생깁니다. <다루는 엔티티, 기본키 타입>을 알려주는 것.
- `findByDiscordId` 처럼 **메서드 이름만 선언하면** 스프링이 SQL을 자동 생성 (5-5의 쿼리 메서드). 구현 클래스도 스프링이 서버 시작 시 자동으로 만들어서 빈으로 등록합니다. 그래서 인터페이스만 있고 구현 코드가 없는 겁니다.
- `Optional<User>` = "결과가 있을 수도, 없을 수도 있다"를 타입으로 표현하는 자바 문법. 없을 때 `null` 대신 빈 상자를 돌려줘서, "없으면 회원가입" 같은 분기를 안전하게 쓸 수 있음.

7-4. DTO 6개 — 데이터 상자

DTO (Data Transfer Object) — 계층 사이 또는 서버 사이에서 데이터를 실어 나르는 전용 상자. 엔티티(DB와 연결됨)를 외부에 직접 노출하면 위험하고 유연성도 떨어져서, 주고받는 모양을 DTO로 따로 정의합니다.

record — 자바의 "값만 담는 불변 클래스" 축약 문법. `public record DiscordLoginRequest(String code) {}` 한 줄이면 생성자, getter, equals 등이 자동 생성됩니다. DTO에 딱 맞음.

```
// Discord의 응답은 snake_case(access_token)인데 자바는 camelCase(accessToken)를 써서
// @JsonProperty로 "JSON의 이 이름을 이 필드에 넣어라"라고 매핑함
@JsonIgnoreProperties(ignoreUnknown = true) // 우리가 선언 안 한 필드가 응답에 있어도 에러 내지 말 것
public record DiscordTokenResponse(
    @JsonProperty("access_token") String accessToken,
    ...
) {}

public record DiscordUserResponse(String id, String username, String avatar, String email) {
    // avatar는 이미지 "해시"값이라 실제 URL로 조합해줘야 함. 아바타 미설정 유저는 null
    public String avatarUrl() {
        if (avatar == null) return null;
        return "https://cdn.discordapp.com/avatars/%s/%s.png".formatted(id, avatar);
    }
}
```


직렬화/역직렬화와 Jackson — 자바 객체 ↔ JSON 문자열 변환을 직렬화/역직렬화라 하고, 스프링에 내장된 Jackson이라는 라이브러리가 자동으로 해줍니다. 컨트롤러가 객체를 반환하면 JSON이 되어 나가는 이유, `@RequestBody` 로 JSON이 객체가 되어 들어오는 이유가 전부 Jackson입니다.

7-5. DiscordOAuthClient.java — Discord와 통신 (흐름의 ④⑤)

```
@Component
public class DiscordOAuthClient {
    private static final String TOKEN_URL = "https://discord.com/api/oauth2/token";
    private static final String USER_URL = "https://discord.com/api/users/@me";

    private final RestClient restClient;
    private final String clientId;
    private final String clientSecret;
    private final String redirectUri;

    public DiscordOAuthClient(
        @Value("${discord.client-id}") String clientId,
        @Value("${discord.client-secret}") String clientSecret,
        @Value("${discord.redirect-uri}") String redirectUri) {
        this.restClient = RestClient.create();
        ...
    }
}
```

코드	의미와 이유
@Component	"이 클래스를 빈으로 등록해줘" (가장 범용적인 빈 등록 어노테이션. @Service, @RestController도 내부적으로는 이것의 변형)
RestClient	스프링이 제공하는 HTTP 요청을 보내는 도구 . 지금까지는 우리가 요청을 "받는" 쪽이었지만, Discord에게는 우리가 클라이언트가 되어 요청을 "보내야" 하므로 필요
@Value("\${discord.client-id}")	설정 파일(application.yaml)의 discord.client-id 값을 생성자 파라미터로 주입 . 코드에 값을 하드코딩하지 않기 위한 장치 (8장 참고)

```

public DiscordTokenResponse exchangeCode(String code) {
    // OAuth2 표준상 토큰 요청은 JSON이 아니라 form-urlencoded 형식이어야 한다
    MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
    form.add("grant_type", "authorization_code");
    form.add("code", code);
    form.add("redirect_uri", redirectUri); // @에서 쓴 값과 일치해야 함 (검증용)
    form.add("client_id", clientId);
    form.add("client_secret", clientSecret);

    try {
        return restClient.post()                // POST 메서드로
            .uri(TOKEN_URL)                    // 이 주소에
            .contentType(MediaType.APPLICATION_FORM_URLENCODED) // form 형식이라고 라벨 붙이고
            .body(form)                        // 본문 싣고
            .retrieve()                        // 보내서 응답 받고
            .body(DiscordTokenResponse.class); // JSON 응답 → 자바 객체로 변환
    } catch (RestClientResponseException e) {
        // code가 만료됐거나, 이미 사용됐거나, redirect_uri가 불일치하는 경우
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED,
            "Discord 인증 코드가 유효하지 않습니다. 다시 로그인해주세요.");
    }
}

```

- 2-2에서 본 "HTTP 요청 3요소"가 코드에 그대로 대응됩니다: `post()` = 메서드, `contentType()` = 헤더, `body()` = 본문.
- `MultiValueMap` = form 형식(`key=value&key=value`) 본문을 만들 때 쓰는 자료구조.
- `try~catch`: Discord가 4xx 에러를 응답하면 `RestClientResponseException` 이 발생 → 그걸 잡아서(catch) 우리 프론트에게는 **401 + 한국어 안내**로 바꿔서 던집니다. `ResponseStatusException` 은 "이 상태코드로 응답을 끝내라"는 스프링 내장 예외입니다.
- `fetchUser()` 는 더 단순합니다: `Authorization: Bearer 토큰` 헤더를 붙여 GET 요청 → 유저 정보 수신.

7-6. AuthService.java — 비즈니스 로직의 본체 (흐름의 ⑥)

```
@Service
@RequiredArgsConstructor
public class AuthService {
    private final DiscordOAuthClient discordOAuthClient; // ㄱ
    private final UserRepository userRepository;          // | 스프링이 자동 주입(DI)
    private final RefreshTokenRepository refreshTokenRepository; // |
    private final JwtProvider jwtProvider;               // ㄴ

    @Value("${jwt.refresh-token-expiration}")
    private long refreshTokenExpirationMs;

    @Transactional
    public LoginResponse loginWithDiscord(String code) {
        // 1. code를 Discord access_token으로 교환
        DiscordTokenResponse discordToken = discordOAuthClient.exchangeCode(code);

        // 2. Discord 유저 정보 조회 (Discord 토큰은 여기까지만 쓰고 버림)
        DiscordUserResponse discordUser = discordOAuthClient.fetchUser(discordToken.accessToken());

        // 3. discordId로 조회 → 있으면 로그인(프로필 갱신), 없으면 자동 회원가입
        User user = userRepository.findByDiscordId(discordUser.id())
            .map(existing -> {
                existing.updateProfile(discordUser.username(), discordUser.avatarUrl());
                return existing;
            })
            .orElseGet(() -> userRepository.save(User.builder()
                .discordId(discordUser.id())
                .username(discordUser.username())
                .avatarUrl(discordUser.avatarUrl())
                .email(discordUser.email())
                .build()));

        // 4. 우리 서비스 토큰 발급
        String accessToken = jwtProvider.createAccessToken(user.getId());
        String refreshToken = issueRefreshToken(user.getId());

        return new LoginResponse(accessToken, refreshToken,
            user.getId(), user.getUsername(), user.getAvatarUrl());
    }
}
```

@Transactional — 이 메서드 안의 DB 작업 전체를 **하나의 묶음(트랜잭션)**으로 처리. 중간에 예외가 나면 그때까지의 DB 변경이 **전부 자동 취소(롤백)**됩니다. 예: 유저 저장은 됐는데 refreshToken 저장에서 터지면 → 유저 저장도 없던 일이 됨. "반쯤 성공한 어정쩡한 상태"를 방지.

3번의 .map().orElseGet() 읽는 법 — Optional의 관용 표현입니다.

"찾았으면(map) 프로필 갱신해서 그대로 쓰고, 못 찾았으면(orElseGet) 새로 만들어 저장해라."

이 한 덩어리가 **자동 회원가입**의 전부입니다. 소셜 로그인에서는 회원가입과 로그인이 사실상 같은 동작입니다.

```
@Transactional
public ReissueResponse reissue(String refreshTokenValue) {
    RefreshToken saved = refreshTokenRepository.findByToken(refreshTokenValue)
        .orElseThrow(() -> new RuntimeException(HttpStatus.UNAUTHORIZED,
            "존재하지 않는 refresh token입니다. 다시 로그인해주세요."));

    if (saved.isExpired()) {
        refreshTokenRepository.delete(saved);    // 만료된 토큰은 삭제하고
        throw new RuntimeException(HttpStatus.UNAUTHORIZED,
            "만료된 refresh token입니다. 다시 로그인해주세요.");    // 재로그인 요구
    }

    String newAccessToken = jwtProvider.createAccessToken(saved.getUserId());
    String newRefreshToken = UUID.randomUUID().toString();
    saved.rotate(newRefreshToken, expiryDate());    // refresh token도 교체 (rotation)

    return new ReissueResponse(newAccessToken, newRefreshToken);
}
```

- 재발급 검증 순서: **DB에 있나? → 만료 안 됐나?** 둘 중 하나라도 실패하면 401 → 프론트는 재로그인 화면으로 보냄.
- "DB에 없음"이 의미하는 것: 이미 로그아웃했거나, 다른 기기 로그인으로 교체됐거나, 위조 시도. 어느 쪽이든 재발급을 거부하는 게 맞습니다.
- `UUID.randomUUID()` = 사실상 중복될 수 없는 랜덤 문자열 생성기. refreshToken은 정보를 담을 필요가 없어서 JWT가 아니라 단순 랜덤값을 씁니다 (검증을 어차피 DB 조회로 하니까).
- `rotate()` 후 `save()` 를 따로 안 부르는 이유: **JPA의 변경 감지(dirty checking)** — @Transactional 안에서 DB에서 꺼낸 엔티티의 값을 바꾸면, 트랜잭션이 끝날 때 JPA가 알아서 UPDATE 쿼리를 날립니다.

7-7. AuthController.java — HTTP 창구

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final AuthService authService;

    @PostMapping("/login/discord")
    public LoginResponse loginWithDiscord(@RequestBody DiscordLoginRequest request) {
        return authService.loginWithDiscord(request.code());
    }

    @PostMapping("/reissue")
    public ReissueResponse reissue(@RequestBody RefreshTokenRequest request) {
        return authService.reissue(request.refreshToken());
    }

    @PostMapping("/logout")
    public void logout(@RequestBody RefreshTokenRequest request) {
        authService.logout(request.refreshToken());
    }
}
```

코드	의미
@RestController	"HTTP 요청을 받는 클래스이고, 반환값은 JSON으로 변환해라"
@RequestMapping("/api/auth")	이 클래스의 모든 API 주소 앞에 공통으로 붙는 경로
@PostMapping("/login/discord")	"POST /api/auth/login/discord 요청이 오면 이 메서드를 실행해라"
@RequestBody	"요청 본문의 JSON을 이 자바 객체(DTO)로 변환해서 넣어달라" (Jackson이 수행)

컨트롤러가 이렇게 짧은 것이 **정상**입니다. HTTP ↔ 자바 변환만 하고, 실제 일은 전부 서비스에 위임합니다.

7-8. JwtProvider.java — JWT 공장

```
@Component
public class JwtProvider {
    private final SecretKey key;
    private final long accessTokenExpirationMs;

    public JwtProvider(
        @Value("${jwt.secret}") String secret,
        @Value("${jwt.access-token-expiration}") long accessTokenExpirationMs) {
        // HS256 서명용 키. secret은 32바이트(글자) 이상이어야 한다.
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.accessTokenExpirationMs = accessTokenExpirationMs;
    }

    public String createAccessToken(Long userId) {
        Date now = new Date();
        return Jwts.builder()
            .subject(String.valueOf(userId)) // 토큰 주인 = 우리 DB의 userId
            .issuedAt(now) // 발급 시각
            .expiration(new Date(now.getTime() + accessTokenExpirationMs)) // 만료 시각
            .signWith(key) // 비밀키로 서명
            .compact(); // 최종 문자열로 조립
    }

    public Long parseUserId(String token) {
        Claims claims = Jwts.parser()
            .verifyWith(key) // 서명 검증 (만료 검사도 자동으로 됨)
            .build()
            .parseSignedClaims(token)
            .getPayload();
        return Long.parseLong(claims.getSubject());
    }
}
```

- `jjwt` 라는 JWT 전문 라이브러리를 씁니다 (팀에서 이미 `build.gradle`에 추가해둬).
- **HS256** = "비밀키 하나로 서명하고, 같은 키로 검증"하는 서명 알고리즘. 키가 32바이트 미만이면 라이브러리가 시작부터 거부합니다.
- `parseUserId()` 가 하는 일 = 4-2의 "서명 검사". 위조됐거나 만료된 토큰이면 `JwtException` 이 던져지고, 정상 이면 내용물(claims)에서 `userId`를 꺼내 돌려줍니다. **DB를 전혀 안 봅니다** — 이게 JWT의 존재 이유.

7-9. JwtAuthenticationFilter.java — 매 요청을 검사하는 문지기

필터(Filter)와 필터 체인 — 스프링에서 모든 HTTP 요청은 컨트롤러에 닿기 전에 여러 개의 필터를 차례로 통과합니다(공항 검색대 줄과 같은 구조... 는 비유니까 건너내면: **요청 전처리 단계들의 연쇄**). Spring Security는 이 체인에 보안 검사 필터들을 끼워 넣는 방식으로 동작하고, 우리는 거기에 JWT 검사 필터를 하나 추가한 것입니다.

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private static final String BEARER_PREFIX = "Bearer ";
    private final JwtProvider jwtProvider;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {

        String header = request.getHeader(HttpHeaders.AUTHORIZATION);

        if (header != null && header.startsWith(BEARER_PREFIX)) {
            String token = header.substring(BEARER_PREFIX.length()); // "Bearer " 잘라내기
            try {
                Long userId = jwtProvider.parseUserId(token);           // 검증 + userId 추출
                var authentication =
                    new UsernamePasswordAuthenticationToken(userId, null, List.of());
                SecurityContextHolder.getContext().setAuthentication(authentication);
            } catch (JwtException | IllegalArgumentException e) {
                // 위조/만료 토큰 → 인증 세팅 없이 통과 (미인증 상태로 처리됨)
            }
        }
        filterChain.doFilter(request, response); // 다음 필터로 넘기기
    }
}
```

- `OncePerRequestFilter` 상속 = "요청당 정확히 한 번 실행되는 필터"를 만드는 표준 방법.
- `SecurityContextHolder` = 스프링 시큐리티가 "현재 요청의 인증 정보"를 담아두는 보관소. 여기에 인증 객체를 넣으면 "이 요청은 userId ○번으로 인증됨"이 되고, 안 넣으면 미인증 상태로 남습니다.
- **토큰이 이상해도 여기서 에러를 내지 않는 이유**: 이 필터는 모든 요청이 지나갑니다. 로그인 API처럼 토큰이 필요 없는 요청도 지나가므로, 여기서는 "인증 세팅"만 하고 **차단 여부의 판단은 다음 단계(SecurityConfig의 규칙)에 맡깁니다**. 보호된 API인데 미인증이면 거기서 401이 납니다.

7-10. SecurityConfig.java — 보안 규칙의 총괄 설정

```
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // (a)
            .cors(cors -> {}) // (b)
            .formLogin(form -> form.disable()) // (c)
            .httpBasic(basic -> basic.disable()) // (c)
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // (d)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll() // (e)
                .requestMatchers("/actuator/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .anyRequest().authenticated() // (f)
            )
            .exceptionHandling(handling -> handling
                .authenticationEntryPoint((request, response, e) ->
                    response.sendError(401)) // (g)
            )
            .addFilterBefore(jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class); // (h)
        return http.build();
    }
}
```

설정		왜
(a)	CSRF 보호 끄	CSRF는 "브라우저가 쿠키를 자동으로 붙이는 것"을 악용하는 공격인데, 우리는 쿠키/세션이 아니라 JWT 헤더 방식이라 공격 자체가 성립 안 함 → 꺼도 안전
(b)	CORS 활성화	아래 7-10-1 참고
(c)	폼 로그인/HTTP Basic 끄	스프링 시큐리티의 기본 로그인 화면·팝업 기능. 우리는 Discord 로그인만 쓰므로 불필요
(d)	세션 안 만들 (STATELESS)	매 요청을 JWT로만 인증하겠다는 선언. 서버가 로그인 상태를 기억하지 않음 (4-1 참고)
(e)	/api/auth/** 전부 허용	로그인 전에 호출하는 API들이므로 토큰 검사 면제. ** 는 "하위 경로 전부"라는 와일드카드
(f)	나머지 전부 인증 필요	기본은 잠그고 예외만 연다 는 보안 원칙. 앞으로 만들 모든 API는 자동으로 보호됨
(g)	미인증 시 401 응답	스프링 기본값은 403인데, "인증 안 됨"의 정확한 코드는 401. 프론트가 401을 보고 재발급/재로그인 로직을 태우므로 중요
(h)	우리 JWT 필터를 체인에 삽입	기존 인증 필터(UsernamePasswordAuthenticationFilter) 앞자리에 끼워 넣음

7-10-1. CORS — 프론트에서 API 호출이 되게 하는 설정

CORS (Cross-Origin Resource Sharing) — 브라우저의 보안 규칙. `localhost:5173` (프론트)에서 `localhost:8080` (백엔드)으로 요청하는 것처럼 **출처(도메인+포트)가 다른 곳으로의 요청을 브라우저가 기본 차단**합니다. 서버가 "이 출처는 허용한다"고 응답 헤더로 알려줘야 브라우저가 통과시킵니다. 그 허용 목록을 정하는 게 아래 설정입니다.

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of(           // 허용할 프론트 주소 목록
        "http://localhost:5173",               // Vite(React) 개발 서버
        "http://localhost:3000"));             // CRA/Next 개발 서버
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    ...
}
```

※ 나중에 실제 프론트 도메인이 생기면 이 목록에 추가해야 합니다. CORS 에러는 백엔드 콘솔엔 안 찍히고 브라우저 콘솔에만 빨강게 나오는 특징이 있어서, 몰라보면 한참 헤매는 단골 이슈입니다.

8. 설정 파일(application.yaml) 해설

application.yaml — 스프링부트의 설정 파일. 코드가 아닌 "값"들(DB 주소, 비밀키, 만료시간...)을 모아두는 곳. **들여쓰기가 곧 문법**이라서, 들여쓰기가 틀리면 설정이 조용히 무시됩니다 (9장에서 실제로 겪음).

```
server:
  error:
    include-message: always # 에러 응답 JSON에 message 필드 포함
                           # (기본값은 숨김이라 "만료된 토큰..." 안내가 프론트에 안 감)

# Discord OAuth2 설정 (시크릿은 환경변수로 주입, git에 올리지 않는다)
discord:
  client-id: ${DISCORD_CLIENT_ID:}
  client-secret: ${DISCORD_CLIENT_SECRET:}
  redirect-uri: ${DISCORD_REDIRECT_URI:http://localhost:5173/auth/callback}

jwt:
  # HS256 서명 키. 32글자 이상 필수. 배포 환경에서는 반드시 환경변수로 교체할 것
  secret: ${JWT_SECRET:steamiron-local-dev-secret-key-change-me-in-production}
  access-token-expiration: 1800000 # 30분 (밀리초)
  refresh-token-expiration: 1209600000 # 14일 (밀리초)
```

`${환경변수명:기본값}` 문법 — "환경변수가 있으면 그 값을, 없으면 콜론 뒤의 기본값을 써라".

`${DISCORD_CLIENT_ID:}` 는 기본값이 빈 문자열 = 환경변수 없이도 서버는 뜨지만 Discord 로그인만 실패.

왜 시크릿을 yaml에 직접 안 적나? — 이 파일은 git에 올라가고, git에 올라간 것은 팀원 전체와 기록에 영원히 남습니다. 시크릿이 유출되면 공격자가 우리 앱 행세를 할 수 있습니다. 그래서 **진짜 값은 각자 PC의 환경변수에 넣고, yaml에는 "환경변수에서 읽어라"**라는 지시만 적습니다. `@Value` 가 이 값을 코드로 가져갑니다 (7-5, 7-8).

9. 구현하면서 실제로 겪은 문제와 해결 과정

문제 1 — 브랜치가 체크아웃이 안 됨

```
$ git checkout feat/discord-login
error: pathspec 'feat/discord-login' did not match any file(s) known to git
```

원인: 브랜치가 원격(GitHub)에 만들어진 시점이 내 마지막 `git fetch` 이후라서, 로컬 git이 브랜치의 존재를 몰랐음.
해결: `git fetch` 후 checkout. **교훈:** 팀원이 새로 푸시한 브랜치로 이동할 땐 fetch 먼저.

문제 2 — yaml 들여쓰기 때문에 JPA 설정이 통째로 무시되고 있었음

# 기존 (잘못됨)	# 수정 후
spring:	spring:
datasource:	datasource:
url: jdbc:postgresql://...	url: jdbc:postgresql://...
jpa: ← datasource의	jpa: ← spring 바로 아래로
hibernate: 자식으로 들어감	hibernate:
ddl-auto: update	ddl-auto: update

영향: `spring.jpa.hibernate.ddl-auto` 가 아니라 `spring.datasource.jpa...` 라는 "존재하지 않는 설정"이 되어 조용히 무시됨 → 테이블 자동 생성이 동작 안 함. **yaml은** 오타가 나도 에러를 안 내고 무시하는 경우가 많아서 특히 위험합니다.

문제 3 — Spring Boot 4에서 RestClient.Builder 빈이 없음

```
APPLICATION FAILED TO START
Parameter 0 of constructor in ...DiscordOAuthClient required a bean of type
'org.springframework.web.client.RestClient$Builder' that could not be found.
```

원인: Spring Boot 3에서는 `RestClient.Builder` 가 자동으로 빈 등록됐지만, Boot 4에서는 구조가 바뀌어 안 됨.
해결: 주입받는 대신 `RestClient.create()` 로 직접 생성. **교훈:** "빈을 찾을 수 없음(required a bean ... could not be found)" 에러는 DI 관련 에러의 대표 형태. 에러 메시지에 어느 클래스의 몇 번째 생성자 파라미터인지까지 다 적혀 있으니 잘 읽으면 됩니다.

문제 4 — 팀 DB에 연결이 안 돼서 서버가 안 뜸

```
Caused by: java.net.SocketTimeoutException: Connect timed out
    at org.postgresql.core.PGStream.createSocket(...)
```

원인: DB 주소 `100.96.97.2` 는 Tailscale(가상 사설망) 주소인데, 이 PC에서 연결 불가 상태였음. JPA는 서버 시작 시점에 DB에 접속하므로 DB가 안 닿으면 서버 자체가 못 뜹니다.
해결(검증용): 10장 참고 — 인메모리 DB로 대체해서 검증.

10. 어떻게 검증했나

"컴파일이 된다"와 "실제로 동작한다"는 다릅니다. 실제 부팅과 API 호출까지 확인했습니다.

1. **컴파일 확인**: `./gradlew :app-api-server:compileJava` — 문법/타입 에러 없음.
2. **부팅 확인**: 팀 DB에 연결이 안 되므로, **H2**(자바 프로세스 안에서 도는 인메모리 DB — 서버 끄면 데이터가 사라져서 테스트에 적합)를 임시 의존성으로 추가하고 환경변수로 `datasource`를 H2로 바꿔치기해서 부팅 → 정상 기동 + `users`, `refresh_tokens` 테이블 자동 생성 확인. **검증 후 H2 의존성은 제거함**.
3. **API 동작 확인** — `curl`(터미널에서 HTTP 요청을 보내는 도구)로 5가지 시나리오 테스트:

테스트	기대	결과	확인된 것
토큰 없이 보호된 URL 호출	401	401 ✓	기본 잠금 + 401 엔트리포인트 동작
actuator health 호출	200	200 ✓	permitAll 예외 규칙 동작
가짜 code로 로그인	401	401 ✓	Discord 호출 → 에러 → 401 변환 경로 동작
모르는 refreshToken으로 재발급	401	401 ✓	"DB에 없으면 거부" 로직 동작
모르는 토큰으로 로그아웃	200	200 ✓	로그아웃은 없는 토큰이어도 조용히 성공

※ 실제 Discord 계정으로 로그인하는 전체 흐름(①~⑥)은 아직 검증 못 함 — Developer Portal 등록(11장)이 선행돼야 함.

11. 남은 할 일 — 직접 해야 하는 것들

1. **Discord Developer Portal** (<https://discord.com/developers/applications>)
 - 애플리케이션 생성 → **OAuth2** 탭에서 Client ID, Client Secret 확인
 - **Redirects**에 `http://localhost:5173/auth/callback` 등록 (프론트 콜백 주소와 일치시킬 것)
2. **환경변수 설정** — `DISCORD_CLIENT_ID`, `DISCORD_CLIENT_SECRET` (IntelliJ: Run Configuration → Environment variables에 `DISCORD_CLIENT_ID=...;DISCORD_CLIENT_SECRET=...`)
3. **팀 DB 연결 확인** — Tailscale 켜져 있는지. DB가 안 닿으면 서버가 안 뜸 (9장 문제 4)
4. **프론트 구현** — ① 로그인 버튼: Discord authorize URL로 이동시키기 ② `/auth/callback` 페이지: URL에서 code 꺼내 백엔드 POST → 받은 토큰을 저장 ③ API 요청마다 `Authorization: Bearer` 헤더 붙이기 ④ 401 받으면 `/reissue` 호출 후 재시도
5. **실제 로그인 전체 흐름 테스트** — 1~4가 끝나면 가능

프론트 구현 시 단골 함정 — React 개발 모드(StrictMode)에서는 `useEffect`가 두 번 실행되어 **code**를 두 번 제출하게 되기 쉽습니다. code는 1회용이라 두 번째 요청은 무조건 401(`invalid_grant`)이 납니다. 콜백 처리가 한 번만 실행되도록 막아야 합니다.

12. 용어 사전 (한 페이지 요약)

용어	한 줄 정의
HTTP	클라이언트와 서버가 주고받는 메시지 형식 규칙. 요청 = 메서드+헤더+본문
GET / POST	조회(데이터는 URL에) / 제출(데이터는 본문에)
헤더	요청/응답에 붙는 이름: 값 형태의 부가 정보
Content-Type	본문이 어떤 표기법(JSON? form?)인지 알려주는 헤더
form-urlencoded	key=value&key=value 표기법. OAuth2 토큰 요청의 필수 형식
상태 코드	응답 결과 요약 숫자. 200 성공, 401 인증 실패, 403 권한 없음, 500 서버 버그
API / 엔드포인트	서버가 공개한 요청 창구 하나하나 (주소+메서드+형식의 약속)
OAuth2	인증을 외부 서비스(Discord)에 위임하는 표준. 비밀번호 대신 증표(code)를 주고받음
code	"방금 Discord 인증에 성공했다"는 1회용·단명 증표
client_id / secret	우리 앱의 이름표(공개 가능) / 우리 앱의 비밀번호(백엔드만)
redirect_uri	인증 후 유저를 돌려보낼 주소. 3곳(포털·①·④)이 정확히 일치해야 함
grant_type	OAuth2 토큰 발급 방식 지정 필드. 우리는 authorization_code
JWT	헤더.내용물.서명 구조의 위조 불가능한 증명서. 서버가 저장 없이 검증 가능
accessToken	매 요청 인증용 JWT. 수명 30분. 무효화 불가라서 짧게
refreshToken	accessToken 재발급 전용. 수명 14일. DB 저장이라 무효화 가능
rotation	재발급 때 refreshToken도 새것으로 교체하는 전략
Bearer	Authorization: Bearer 토큰 — 토큰을 싣는 관용 표기
Spring Boot	자바 서버 프레임워크(Spring)의 설정을 자동화한 것
어노테이션(@)	스프링 등이 읽는 라벨. "이 클래스는 이렇게 취급해라"
빈(Been)	스프링이 만들어 보관·관리하는 객체
DI (의존성 주입)	필요한 빈을 스프링이 생성자에 자동으로 넣어주는 것
Controller/Service/Repository	HTTP 창구 / 비즈니스 로직 / DB 접근 — 3계층 분리
DTO / record	데이터 전달 전용 상자 / 그걸 한 줄로 만드는 자바 문법
JPA / Hibernate	자바 객체 ↔ DB 테이블 자동 변환 기술 (표준 / 구현체)
엔티티	DB 테이블과 1:1 대응하는 자바 클래스 (@Entity)
쿼리 메서드	findByDiscordId 처럼 메서드 이름으로 SQL을 자동 생성하는 기능
@Transactional	메서드 내 DB 작업을 한 묶음으로. 실패 시 전부 롤백
변경 감지	트랜잭션 안에서 엔티티 값을 바꾸면 JPA가 UPDATE를 자동 실행
필터 체인	요청이 컨트롤러 도달 전 통과하는 전처리 단계들. 시큐리티의 동작 방식

SecurityContextHolder	현재 요청의 인증 정보 보관소
STATELESS	서버가 세션(로그인 상태)을 저장하지 않는 방식. JWT로만 인증
CSRF	쿠키 자동 전송을 악용하는 공격. 쿠키를 안 쓰는 우리는 보호 기능 꺼도 안전
CORS	다른 출처(포트 포함)로의 브라우저 요청 차단 규칙. 서버가 허용 목록을 응답해야 통과
RestClient	스프링에서 HTTP 요청을 "보낼" 때 쓰는 도구
@Value / \${ENV:기본값}	yaml·환경변수 값을 코드로 주입 / 환경변수 없으면 기본값 쓰는 문법
ddl-auto: update	서버 시작 시 엔티티 보고 테이블 자동 생성/수정 (개발용)
H2	프로세스 안에서 도는 인메모리 DB. 테스트·검증용
UUID	사실상 중복 불가능한 랜덤 문자열. refreshToken 값으로 사용